SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

Questions: 5 **Duration:** 1 Hour **Total Marks:** 50

Annexure A - Analytical Problem Solving

Name	R. Kishan
Register No.	192525378

Code of Conduct:

I R. Kishan / 192525378 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1: Water Jug Problem

Problem understanding. Two jugs: 4-gallon (X) and 3-gallon (Y), no markings, one pump. Goal: exactly 2 gallons in the 4-gallon jug.
Formulation. State = (x, y), where x = water in the 4-gal jug (0-4) and y = water in the 3-gal jug (0-3). Initial state (0, 0). Goal state (2, n) for any n. Path cost = 1 per action.
Method. Uninformed search (BFS) over the state space using the production rules below, so the shortest sequence is found.

Rule	Condition	Action (new state)
1. Fill X	$x < 4$	(4, y)
2. Fill Y	$y < 3$	(x, 3)
3. Empty X	$x > 0$	(0, y)
4. Empty Y	$y > 0$	(x, 0)
5. Pour X into Y until Y full	$x + y \geq 3, x > 0$	(x - (3 - y), 3)
6. Pour Y into X until X full	$x + y \geq 4, y > 0$	(4, y - (4 - x))
7. Pour all of X into Y	$x + y \leq 3, x > 0$	(0, x + y)
8. Pour all of Y into X	$x + y \leq 4, y > 0$	(x + y, 0)

Solution (shortest, 6 steps):

Step	State (x, y)	Rule applied
0	(0, 0)	Initial state
1	(0, 3)	Fill the 3-gal jug (Rule 2)
2	(3, 0)	Pour 3-gal jug into 4-gal jug (Rule 8)
3	(3, 3)	Fill the 3-gal jug again (Rule 2)
4	(4, 2)	Pour from 3-gal into 4-gal until 4-gal is full; 1 gal goes in, 2 gal remain (Rule 6)
5	(0, 2)	Empty the 4-gal jug onto the ground (Rule 3)
6	(2, 0)	Pour the 2 gal from 3-gal jug into 4-gal jug (Rule 8). **Goal reached**

Result. The 4-gallon jug contains exactly 2 gallons after 6 actions. An alternative 6-step path is (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3), where the 4-gal jug holds 2 gal at the last state.

Question 2: Mars Rover (PEAS Analysis)

A Mars rover is an autonomous agent that moves over Martian terrain, collects soil and rock samples, analyses them with on-board instruments and sends the data to Earth.

i. Percepts

Camera images (stereo/panoramic), terrain slope and obstacle data from LIDAR/hazard cameras, wheel odometry and IMU/gyroscope readings, GPS-substitute localisation (sun sensor, star tracker), spectrometer and X-ray/chemical sensor readings of samples, temperature, wind and dust sensors, battery/solar power level, wheel-motor current, communication signals/commands from Earth.

ii. Operating environment

Property	Mars rover environment	Reason
Observability	Partially observable	Sensors see only nearby terrain; the soil below the surface is unknown
Agents	Single agent	Other rovers act independently and are not part of its decisions
Determinism	Stochastic	Wheel slip, dust storms, uncertain terrain and sensor noise
Episodic/Sequential	Sequential	Each move changes later positions, power and options
Static/Dynamic	Dynamic	Dust storms, lighting and temperature change while it acts
Discrete/Continuous	Continuous	Positions, speeds and sensor values are continuous
Known/Unknown	Partly unknown	The terrain map and sample properties are discovered while exploring

iii. Actions (actuators)

Move forward/backward, turn, stop; climb or avoid slopes and obstacles; deploy robotic arm; drill/scoop/grind rock and soil; place sample in analyser (spectrometer, camera, etc.); take photographs; raise/lower mast and rotate antenna; transmit data to Earth; switch to sleep/charge mode, heat or cool instruments.

iv. Performance measure

Number and scientific value of samples collected and analysed; area explored; accuracy of the analysis and data returned to Earth; safety (no tipping, getting stuck or damage); energy efficiency and mission lifetime; time taken to reach the target sites.

v. Most suitable agent architecture

A **utility-based agent with learning** (hybrid, with a model-based reflex layer for safety) is most suitable. Reasons: (a) the environment is partially observable, so the rover must keep an internal model of the map and its own state; (b) it has several competing goals (science value, power, safety, time), so it needs a utility function to trade them off rather than a single yes/no goal; (c) simple reflex rules (stop at cliff, tilt protection) are still needed for fast, safe reactions because communication with Earth is delayed 4-24 minutes; (d) a learning component improves terrain classification and path choices over time.

Question 3: 8-Queens Problem

Problem understanding. Place 8 queens on an 8 × 8 board so that no two queens share a row, column or diagonal. The configuration given in the question is not a solution because the queens in the first and last columns lie on the same diagonal (a diagonal attack), so the goal test fails for it.

Problem formulation (incremental formulation).

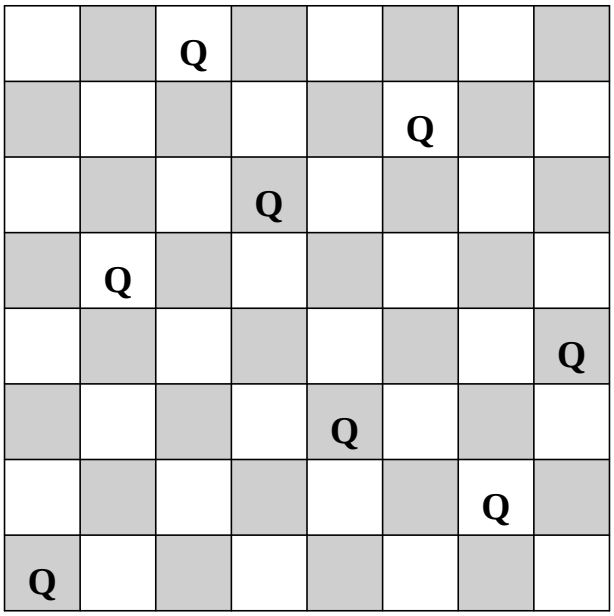
Component	Definition
Initial state	Empty board (no queens placed)
States	Any arrangement of 0 to 8 queens on the board, with queens added column by column from left to right and none attacking another (about 2,057 states)
Actions	Place a queen in the leftmost empty column, in any row that is not attacked by queens already on the board
Transition model	Returns the board with the new queen added in the chosen square
Goal test	8 queens on the board and no queen attacks another (same row, column or diagonal)
Path cost	Not important: only the final arrangement matters (cost 1 per placement if needed, so every solution costs 8)

Search space and strategies. A naive complete-state formulation (any 8 queens on 64 squares) has 64C8 ≈ 4.4×10^9 states; restricting to one queen per column gives $8^8 \approx 16.7$ million; one queen per row *and* column gives 8! = 40,320 permutations; the incremental formulation above reduces the space to about 2,057 states, which is why it is preferred.

Uninformed strategies: BFS works but stores very many partial boards. **Depth-first search with backtracking** is best: the solution lies at a fixed depth (8), memory is only O(8), and when a column has no safe row the search backtracks to the previous column and tries the next row.

Applying it to the given configuration. Since the first and last column queens attack diagonally, DFS would backtrack to the last queen, move it to another safe row (or if none, move the previous one) until no attacks remain. Constraint check for a queen at (row r, column c): another queen at (r2, c2) attacks it if r = r2, or |r - r2| = |c - c2|.

One valid solution (row of the queen in columns 1 to 8 = 1, 5, 8, 6, 3, 7, 2, 4); the problem has 92 solutions (12 distinct up to symmetry):



Check: all rows are different, all columns are different, and the values (row - column) and (row + column) are all distinct, so no diagonal attacks exist.

Question 4: OLA Cab Booking Agent

Problem understanding. The customer gives a pickup location and a destination; the app must choose a cab type (Micro, Mini, Sedan, Prime, Shared, ...) and a route that best matches the customer's comfort, cost and time.

Type of agent. A **goal-based problem-solving agent** that is refined into a **utility-based agent**. The goal is 'reach the destination'. Because there are several ways (cab types and routes) that all reach the goal but differ in fare, comfort, waiting time and travel time, a utility function ranks them and picks the best. The environment is observable (map, traffic, fares and cab availability come from the server), single agent from the user's view, dynamic (traffic, fares and availability change) and sequential.

Problem formulation. Initial state: customer at pickup location. Actions: choose cab type, confirm booking, drive along road segments. Goal test: customer reached the destination. Path cost: fare + travel time (+ waiting time, discomfort penalty).

Pseudocode

```
function OLA-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty), state, goal, problem

  state <- UPDATE-STATE(state, percept)       // location, time, traffic, fares
  if seq is empty then
      goal    <- FORMULATE-GOAL(state)        // reach destination D from pickup P
      problem <- FORMULATE-PROBLEM(state, goal)
      seq     <- SEARCH-AND-CHOOSE(problem)
      if seq = failure then return NO-CAB-AVAILABLE
  action <- FIRST(seq);  seq <- REST(seq)
  return action

function SEARCH-AND-CHOOSE(problem) returns a plan
  options <- empty list
  for each cab_type in {Micro, Mini, Sedan, Prime, Shared, ...} do
      if AVAILABLE(cab_type, problem.pickup) then
          route <- BEST-ROUTE(problem.pickup, problem.dest)  // A* / UCS
          fare  <- ESTIMATE-FARE(cab_type, route)
          time  <- ETA(cab_type, route) + WAIT-TIME(cab_type)
          U     <- w1*Comfort(cab_type) - w2*fare - w3*time    // user preferences
          add (cab_type, route, U) to options
  if options is empty then return failure
  best <- option in options with maximum U
  return [SELECT(best.cab_type), CONFIRM-BOOKING, ASSIGN-DRIVER,
          TRACK-TRIP(best.route), END-TRIP-AND-PAY]

// Main loop
repeat
    percept <- GET-USER-INPUT()          // pickup, destination, preferences
    action  <- OLA-PROBLEM-SOLVING-AGENT(percept)
    EXECUTE(action)
until destination reached
```

Result. The agent returns the cab type and route with the highest utility, then executes booking and trip actions until the goal test (destination reached) is true.

Question 5: Uniform Cost Search (UCS)

Problem understanding. Find the least-cost path from S to G in the weighted directed graph. Edges (cost): S-A 1, S-G 12, A-B 3, A-C 1, B-D 3, C-D 1, C-G 2, D-G 3.
Method. UCS expands the frontier node with the lowest path cost g(n) using a priority queue; it is complete and optimal when all step costs are positive. The goal test is applied when a node is *removed* from the queue, so the first time G is selected the path is optimal.

Step-by-step trace (frontier shown as node:cost, with parent):

Step	Node expanded	Frontier (priority queue) after expansion	Explored
1	S (0)	A:1 (S), G:12 (S)	S
2	A (1)	C:2 (A), B:4 (A), G:12 (S)	S, A
3	C (2)	D:3 (C), B:4 (A), G:4 (C), G:12 replaced by 4 via C	S, A, C
4	D (3)	B:4 (A), G:4 (C). G via D = 6 > 4, ignored	S, A, C, D
5	B (4)	G:4 (C). D via B = 7 > 3, ignored	S, A, C, D, B
6	G (4)	Goal test succeeds on removal of G	S, A, C, D, B, G

Final path: S → A → C → G, with total cost = 1 + 1 + 2 = **4**.
Comparison of the other paths:

Path	Cost
S-A-C-G	1 + 1 + 2 = 4 (optimal)
S-A-C-D-G	1 + 1 + 1 + 3 = 6
S-A-B-D-G	1 + 3 + 3 + 3 = 10
S-G (direct)	12

Interpretation. Although the direct route S-G has the fewest hops, it is the most expensive (12 units of fuel + time). The delivery should use S, A, C, G, with a minimum cost of 4 units, which saves 8 units (about 67%) compared with the direct route. UCS guarantees this is the optimal path because it always expands the cheapest node first and all costs are positive. Its time and space complexity is $O(b^{1+\text{floor}(C^*/e)})$, where C* is the optimal cost and e the minimum edge cost.